

Judging Notes

1. Judge System

When you submit source code, the judge system determines a verdict by compiling and running it against undisclosed test inputs.

1.1. Input and Output

- Your program must read all input from **standard input** (stdin, e.g., `std::cin` in C++, `System.in` in Java, `sys.stdin` in Python).
- Your program must write all its output to **standard output** (stdout, e.g., `std::cout` in C++, `System.out` in Java, `sys.stdout` in Python).
- Output written to **standard error** (stderr, e.g., `std::stderr` in C++, `System.error` in Java, `sys.stderr` in Python) is ignored when judging.
- Your program is not allowed to open, create, or write to files. These file operations will result in a **RUN-ERROR**.
- Your program must exit with status code 0. Exiting with a non-zero status code will result in a **RUN-ERROR**, even if the output is correct.

1.2. Execution Environment (Sandbox)

Your program will be run inside a *sandboxed environment*. This environment isolates the run of your program to prevent system damage and to ensure equal conditions for all contestants. This means several restrictions apply:

- **Memory Usage:** Total memory usage is limited to **2 GB**. This limit applies to the entire processes, including your program's code, static variables, stack, heap, and any virtual machine overhead (like for Java or Python). Exceeding this limit will typically result in a **RUN-ERROR**.
- **Stack Size:** In C/C++, the stack size is not explicitly limited, but is limited by the total memory limit. In Java/Kotlin, the stack size is limited by the `-Xss` command line parameter. In Python, the stack size is limited by the Python runtime's default limit. Your program can attempt to change it by calling `sys.setrecursionlimit()`, but the stack size is still ultimately limited by the total memory limit.
- **Processes/Threads:**
 - Your program will be run on a **single processor core**. Using multiple threads or processes is allowed but generally discouraged as it will not likely improve performance.
 - The total number of processes allowed for a run is limited to **64**. This includes the main process of your program and any child processes, as well as processes the system might use internally. Exceeding this limit will result in a **RUN-ERROR**.
- **External Commands:** Running external commands (e.g., via `system()` in C/C++) is restricted and will not likely work as expected.
- **File System Access:** As mentioned, opening, creating or writing to files is not allowed.

1.3. Submission and Compilation

Your submission must meet these requirements:

- **Submission Files:** A single submission can contain one or more files. Each file name must start with an ASCII alphanumeric character (a "half-width" `A-Z`, `a-z`, or `0-9`) and may contain only ASCII alphanumeric characters and `+.\\_-`.
- **Source Code Size:** The total size of all source code files in a single submission must not exceed **256 KB**.
- **Source Code Encoding:** All source code files must be encoded in UTF-8. Submitting files with other encodings may result in a **COMPILER-ERROR** or unexpected behavior.
- **Compilation Time:** Your submission must compile successfully within **30 seconds**. A compilation taking longer than this will be aborted, resulting in a **COMPILER-ERROR**.
- **Compilation Memory Usage:** Total memory usage during compilation is limited to **2 GB**. A compilation exceeding this will be aborted, resulting in a **COMPILER-ERROR**.

1.4. About the Platform

The judge system runs your program on Google Compute Engine, C4 machine type (`c4-highcpu-2`, with `turboMode` set as `ALL_CORE_MAX`). For more information about Google Compute Engine, please visit the official website¹.

1. <https://cloud.google.com/compute/docs/cpu-platforms>

2. Compilers & Options

The judge system uses the following compilers and execution environments (e.g., interpreters) with the following options. "\$@" is substituted with your source file name(s); "\$DEST" is the name of the binary and is chosen by the system.

The **Run** commands in the following table are for non-interactive problems. For interactive problems, standard input and output are connected to a judge program. See the "Note on Interactive Problems" section below for the details.

C	
Version	gcc 13.3.0
Compile	gcc -x c -g -O2 -std=gnu11 -static -o "\$DEST" "\$@" -lm
Run	"\$DEST" < <i>infile</i> > <i>outfile</i>
C++	
Version	g++ 13.3.0
Compile	g++ -x c++ -g -O2 -std=gnu++20 -static -o "\$DEST" "\$@"
Run	"\$DEST" < <i>infile</i> > <i>outfile</i>
Java	
Version	OpenJDK 21.0.8
Compile	javac -encoding UTF-8 -sourcepath . -d . "\$@"
Run	java -Dfile.encoding=UTF-8 -XX:+UseSerialGC -Xss64m -Xms1920m -Xmx1920m <i>MainClass</i> < <i>infile</i> > <i>outfile</i>
Python 3 (PyPy)	
Version	Python 3.11.11 (PyPy 7.3.18)
Compile	pypy3 -m py_compile "\$@"
Run	pypy3 "\$@" < <i>infile</i> > <i>outfile</i>
Kotlin	
Version	1.9.24
Compile	kotlinc -d . "\$@"
Run	kotlin -Dfile.encoding=UTF-8 -J-XX:+UseSerialGC -J-Xss64m -J-Xms1920m -J-Xmx1920m <i>MainClass</i> < <i>infile</i> > <i>outfile</i>

For Java and Kotlin submissions, you do not need to name your main class `Main`. When you submit, the web interface tries to auto-detect the main class. Double-check the detected class name before submitting multiple files.

In Python, **Compile** commands only verify the syntax using the `py_compile` module. `*.pyc` files will not be used in the actual run.

The compilers and the execution environments are also available on your machine, as the following commands:

- **C** — `compilegcc / runc`
- **C++** — `compileg++ / runcpp`
- **Java** — `compilejava / runjava`
- **Python 3** — `compilepython3 / runpython3`
- **Kotlin** — `compilekotlin / runkotlin`

3. Verdicts of Submissions

The judge system runs your program against multiple inputs prepared by the judges.

The system first decides the verdicts of your submission for each input. Then, based on those verdicts, the system decides an overall single verdict of your submission. You will only see this overall verdict, not the verdicts for each input.

3.1. Possible Verdicts on Each Input

For each input, the judge system decides one of the following verdicts:

- **CORRECT:** The judge system determined that your program completed its run within the time limit and produced an output that was validated as correct.
 - The validation method is prepared by the judges. This may involve comparing the output to a specific answer or using a custom program for problems where the correct output is not unique.
 - Minor differences in whitespaces are ignored. For example, an extra whitespace at the end of a line or an extra blank line at the end of the output can be ignored.
 - Differences between uppercase and lowercase letters are ignored. For example, "Yes" and "yes" are treated the same.
 - For problems where the correct output may not be unique (e.g., with floating-point answers), the specific validation method will be documented in the problem statement.
- **TIMELIMIT (Time Limit Exceeded):** Your program did not complete the run within the per-problem time limit and was aborted.
- **RUN-ERROR (Runtime Error):** Your program terminated abnormally during the run. Common causes include:
 - Attempting illegal operations (e.g., division by zero).
 - Accessing invalid memory (e.g., array index out of bounds, null pointer dereferencing).
 - Exceeding the memory limit (2 GB).
 - Aborting by `assert()`.
 - Exiting with a non-zero status code (e.g., forgetting `return 0;` at the end of `main` in C). The exit status code is checked *before* the output is checked.
 - Crashing for any other reason.
- **WRONG-ANSWER:** Your program completed the run within the time limit without crashing, but produced an output that was not validated as correct.
- **OUTPUT-LIMIT (Output Limit Exceeded):** Your program produced an output exceeding **8 MB**. This limit is not applied to output written to standard error.
- **NO-OUTPUT:** Your program completed the run successfully within the time limit without crashing but did not produce any output on standard output.

3.2. Overall Verdict of a Submission

The judge system decides one overall verdict of a submission as follows:

3.2.1. Accepted

- **CORRECT**: Your program resulted in **CORRECT** for all inputs.

3.2.2. Rejected (with 20 Minute Penalty)

If the system decides any of the five verdicts below for an input, the decided verdict will be the overall verdict of the submission, and it will incur a **20-minute time penalty** per submission if the problem is eventually solved.

- **TIMELIMIT**
- **RUN-ERROR**
- **WRONG-ANSWER**
- **OUTPUT-LIMIT**
- **NO-OUTPUT**

If multiple inputs result in different verdicts including ones from this list (e.g., one **TIMELIMIT** and one **WRONG-ANSWER**), the system decides one of the verdicts from the list as the overall verdict of the submission. In this case, there's no guarantee which one is selected.

3.2.3. Rejected (with No Penalty)

Two verdicts below mean that the system did not run your program against the inputs. They **do not incur any time penalty**.

- **COMPILER-ERROR**: The judge system failed to compile your source code. You can see the compiler's error messages on the submission details page.
- **TOO-LATE**: The judge system received your submission after the contest ends. Note that submissions received before the contest ends will be judged even after the contest ends, and the system decides a verdict that is different from **TOO-LATE**.

4. Note on Interactive Problems

You may meet “interactive problems” in the contest. Like standard problems, your program will read from standard input and write to standard output. The key difference is that the standard input and output are connected to a special judge program, requiring your program to communicate back and forth with it. In contrast to standard problems where the input is fixed, the input in an interactive problem may change based on the outputs that your program previously wrote.

In most programming environments, program output is buffered to enhance I/O performance. For interactive problems, your output must be sent from your program immediately, rather than being held in these internal buffers. This typically means you must explicitly **flush** the output buffer after each piece of information you write to the standard output.

- C/C++ (`stdio.h` / `cstdio`): Use `fflush(stdout);`. Writing `\n` does not guarantee a flush.
- C++ (`iostream`): Writing `std::endl` automatically flushes the output stream (e.g., `std::cout`) upon writing a newline. If you output a newline using `\n` instead (which does not guarantee a flush), explicitly call `std::cout.flush();`.
- Java / Kotlin: `System.out` is configured for “auto-flush” by default, meaning `println()` flushes the buffer upon writing a newline. When using other streams, invoke the `flush()` method on your output stream or writer.
- Python: Use `sys.stdout.flush()` after your `print()` statements. Alternatively, you can use the `flush=True` argument: `print(..., flush=True)`.

The time limit for an interactive problem is how much time your submission spend; the time spent by the judge program is *not* counted towards this. Note that if your program attempts to read more input than can be provided by the judge program at that time (e.g., because you forgot to flush your previous output, or because of some other reason), then your program will stall indefinitely and your submission will result in a **TIMELIMIT**.

5. Note on Languages

The judges have solved all problems in languages from at least two of the three distinct language groups (Java/Kotlin, C/C++, and Python).

6. Note to Python Users

Only syntax errors will be reported as a **COMPILER-ERROR**. Other errors, such as `NameError` or `ModuleNotFoundError`, will result in a **RUN-ERROR** and incur a 20-minute penalty.

It is fine, though not needed, to start your scripts with an interpreter directive (line starting with `#!`, also known as a shebang).^{*2}

2. Some past versions of DOMjudge refused scripts that contain a shebang.

6.1. Available Python Modules

<code>_decimal</code>	<code>_pypy_winbase_cffi</code>	<code>dis</code>	<code>pyexpat</code>
<code>_exceptions__</code>	<code>_pypy_winbase_cffi64</code>	<code>distutils</code>	<code>pypy_tools</code>
<code>_future__</code>	<code>_pypyjson</code>	<code>doctest</code>	<code>pypyjit</code>
<code>_hello__</code>	<code>_random</code>	<code>email</code>	<code>pyrepl</code>
<code>_phello__</code>	<code>_rawffi</code>	<code>encodings</code>	<code>queue</code>
<code>_pypy__</code>	<code>_resource_build</code>	<code>ensurepip</code>	<code>quopri</code>
<code>_abc</code>	<code>_resource_cffi</code>	<code>enum</code>	<code>random</code>
<code>_aix_support</code>	<code>_scproxy</code>	<code>errno</code>	<code>re</code>
<code>_ast</code>	<code>_sha1</code>	<code>faulthandler</code>	<code>readline</code>
<code>_audioop_build</code>	<code>_sha256</code>	<code>fcntl</code>	<code>reprlib</code>
<code>_audioop_cffi</code>	<code>_sha3</code>	<code>filecmp</code>	<code>resource</code>
<code>_blake2</code>	<code>_sha512</code>	<code>fileinput</code>	<code>rlcompleter</code>
<code>_bootsubprocess</code>	<code>_signal</code>	<code>fnmatch</code>	<code>runpy</code>
<code>_bz2</code>	<code>_sitebuiltins</code>	<code>fractions</code>	<code>sched</code>
<code>_cffi_backend</code>	<code>_socket</code>	<code>ftplib</code>	<code>secrets</code>
<code>_cffi_ssl</code>	<code>_sqlite3</code>	<code>functools</code>	<code>select</code>
<code>_codecs</code>	<code>_sqlite3_build</code>	<code>future_builtins</code>	<code>selectors</code>
<code>_codecs_cn</code>	<code>_sqlite3_cffi</code>	<code>gc</code>	<code>shelve</code>
<code>_codecs_hk</code>	<code>_sre</code>	<code>genericpath</code>	<code>shlex</code>
<code>_codecs_iso2022</code>	<code>_ssl</code>	<code>getopt</code>	<code>shutil</code>
<code>_codecs_jp</code>	<code>_ssl_build</code>	<code>getpass</code>	<code>signal</code>
<code>_codecs_kr</code>	<code>_string</code>	<code>gettext</code>	<code>site</code>
<code>_codecs_tw</code>	<code>_strptime</code>	<code>glob</code>	<code>smtpd</code>
<code>_collections</code>	<code>_struct</code>	<code>graphlib</code>	<code>smtplib</code>
<code>_collections_abc</code>	<code>_structseq</code>	<code>greenlet</code>	<code>sndhdr</code>
<code>_colorize</code>	<code>_sysconfigdata</code>	<code>grp</code>	<code>socket</code>
<code>_compat_pickle</code>	<code>_syslog_build</code>	<code>gzip</code>	<code>socketserver</code>
<code>_compression</code>	<code>_syslog_cffi</code>	<code>hashlib</code>	<code>sqlite3</code>
<code>_contextvars</code>	<code>_testcapi</code>	<code>heapq</code>	<code>sre_compile</code>
<code>_continuation</code>	<code>_testing</code>	<code>hmac</code>	<code>sre_constants</code>
<code>_cppyy</code>	<code>_testmultiphase</code>	<code>html</code>	<code>sre_parse</code>
<code>_crypt</code>	<code>_testmultiphase_build</code>	<code>http</code>	<code>ssl</code>
<code>_csv</code>	<code>_thread</code>	<code>identity_dict</code>	<code>stackless</code>
<code>_ctypes</code>	<code>_threading_local</code>	<code>idlelib</code>	<code>stat</code>
<code>_ctypes_test</code>	<code>_tkinter</code>	<code>imaplib</code>	<code>statistics</code>
<code>_ctypes_test_build</code>	<code>_vmprof</code>	<code>imghdr</code>	<code>string</code>
<code>_curses</code>	<code>_warnings</code>	<code>imp</code>	<code>stringprep</code>
<code>_curses_build</code>	<code>_weakref</code>	<code>importlib</code>	<code>struct</code>
<code>_curses_cffi</code>	<code>_weakrefset</code>	<code>inspect</code>	<code>subprocess</code>
<code>_curses_panel</code>	<code>_winapi</code>	<code>io</code>	<code>sunau</code>
<code>_dbm</code>	<code>abc</code>	<code>ipaddress</code>	<code>syntable</code>
<code>_decimal_build</code>	<code>aifc</code>	<code>itertools</code>	<code>sys</code>
<code>_ffi</code>	<code>antigravity</code>	<code>json</code>	<code>sysconfig</code>
<code>_gdbm</code>	<code>argparse</code>	<code>keyword</code>	<code>syslog</code>
<code>_gdbm_build</code>	<code>array</code>	<code>lib2to3</code>	<code>tabnanny</code>
<code>_gdbm_cffi</code>	<code>ast</code>	<code>linecache</code>	<code>tarfile</code>
<code>_hashlib</code>	<code>asynchat</code>	<code>locale</code>	<code>telnetlib</code>
<code>_hpy_universal</code>	<code>asyncio</code>	<code>logging</code>	<code>tempfile</code>
<code>_immutables_map</code>	<code>asyncore</code>	<code>lzma</code>	<code>termios</code>
<code>_imp</code>	<code>atexit</code>	<code>mailbox</code>	<code>test</code>
<code>_io</code>	<code>audioop</code>	<code>mailcap</code>	<code>textwrap</code>
<code>_jitlog</code>	<code>base64</code>	<code>marshal</code>	<code>this</code>
<code>_locale</code>	<code>bdb</code>	<code>math</code>	<code>threading</code>
<code>_lsprof</code>	<code>binascii</code>	<code>mimetypes</code>	<code>time</code>
<code>_lzma</code>	<code>bisect</code>	<code>mmap</code>	<code>timeit</code>
<code>_lzma_build</code>	<code>builtins</code>	<code>modulefinder</code>	<code>tkinter</code>
<code>_lzma_cffi</code>	<code>bz2</code>	<code>msilib</code>	<code>token</code>
<code>_markupbase</code>	<code>cProfile</code>	<code>msvcrt</code>	<code>tokenize</code>
<code>_marshal</code>	<code>calendar</code>	<code>multiprocessing</code>	<code>tomllib</code>
<code>_md5</code>	<code>cffi</code>	<code>netrc</code>	<code>tput</code>

_minimal_curses	cgi	nntplib	trace
_multibytecodec	cglib	ntpath	traceback
_multiprocessing	chunk	nturl2path	tracemalloc
_opcode	cmath	numbers	tty
_operator	cmd	opcode	turtle
_osx_support	code	operator	turtledemo
_overlapped	codecs	optparse	types
_pickle_support	codeop	os	typing
_posixshm	collections	pathlib	unicodedata
_posixshm_build	colorsys	pdb	unittest
_posixshm_cffi	compileall	pickle	urllib
_posixsubprocess	concurrent	pickletools	uu
_pwdgrp_build	configparser	pipes	uuid
_pwdgrp_cffi	contextlib	pkgutil	venv
_py_abc	contextvars	platform	warnings
_pydecimal	copy	plistlib	wave
_pyio	copyreg	poplib	weakref
_pypy_generic_alias	cpyext	posix	webbrowser
_pypy_interact	crypt	posixpath	wsgiref
_pypy_irc_topic	csv	pprint	xdrlib
_pypy_openssl	ctypes	profile	xml
_pypy_remote_debug	ctypes_support	pstats	xmlrpc
_pypy_testcapi	curses	pty	zipapp
_pypy_util_build	dataclasses	pwd	zipfile
_pypy_util_cffi	datetime	py_compile	zipimport
_pypy_util_cffi_inner	dbm	pycbr	zlib
_pypy_wait	decimal	pydoc	zoneinfo
_pypy_winbase_build	difflib	pydoc_data	